



Towards the Estimation of Quality Attributes on System Model Histories

Konstantin Rupert Blaschke

fortiss, Munich, Germany

Blekinge Institute of Technology, Karlskrona, Sweden

blaschke@fortiss.org

Simon Barner

fortiss

Munich, Germany

barner@fortiss.org

Abstract

Companies increasingly rely on Model-Based Systems Engineering to develop Cyber-Physical Systems such as cars, aircraft, or medical devices. The quality of engineering model artifacts is key to efficient collaboration in systems engineering with multi-tier supply chains. Ensuring model artifact quality and comprehensibility for practitioners is challenging. Manual reviews are time- and cost-intensive and subject to bias, whereas existing automated methods based on syntactical rules and model metrics are limited in scope. The paper presents work towards swift quality feedback to system engineers during modeling. The concept allows domain and project-specific context and is applicable to industry-size model artifacts. We implement a data-driven estimation that combines automated model metric extraction with expert quality assessments. We leverage the system model version history from an open-source miniature automotive demonstrator. We assess the model versions' comprehensibility and showcase a semi-automated pipeline to initiate a model quality estimator. We achieve an average accuracy of 0.94 with a random forest approach on our test data.

CCS Concepts

• **Software and its engineering** → **Model-driven software engineering**; • **Computer systems organization** → *Embedded software*; • **General and reference** → *Metrics*.

Keywords

Model-based Systems Engineering, Model Quality, Model Metrics, Quality Assessment, Model Review

ACM Reference Format:

Konstantin Rupert Blaschke and Simon Barner. 2024. Towards the Estimation of Quality Attributes on System Model Histories. In *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS Companion '24)*, September 22–27, 2024, Linz, Austria. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3652620.3688339>

1 Introduction

The complexity of developing Cyber-Physical Systems (CPS) increases with market demands and regulations. CPSs such as cars,

aircraft, or medical devices, react to swift environmental changes, following strict safety requirements [7, 28, 39].

CPS development implies a high degree of cross-domain collaboration [8]. Model-Based Systems Engineering (MBSE), a foundational principle for developing CPSs, supports engineers in complexity reduction and collaboration. System complexity also comes with new requirements for MBSE tools and languages.

Recent trends in the industry highlight the need for improved precision, consistency, integration, and interoperability of engineering modeling languages and tools [30]. The improvements are critical for domain-specific applications within MBSE [16].

Despite tooling advancements, the systems engineering community still struggles to maintain the quality of system model artifacts [3, 16]. Collaborations of modelers from different domains (e.g., software, electronics, construction, ...), product extensions, and renewed requirements foster re-engineering and re-design. Collaborators need to comprehend models easily to contribute to shared system model artifacts, which demands high model quality. Cross-domain collaboration in complex systems often leads to the opposite: a decrease in model quality. The reasons are varying levels of experience and unfamiliarity with methodologies. Even within the same company, cross-team modeling with increasing complexity, artifact size, and depth, can lead to inconsistencies and lack of component reuse [27, 40].

As in code-based projects, technical debt is also introduced over time in model-based projects through clones, inappropriate abstraction levels, or inconsistent naming. Technical debt further results in maintainability and extensibility issues in system model development [36], which are typically linked to low comprehensibility. Therefore, assessing comprehensibility is one step towards maintaining high quality in system model artifacts and towards successfully implementing and leveraging the benefits of MBSE. As Liu *et al.* point out, maintainability, comprehensibility, and extensibility will benefit from high quality [25].

We focus on higher-level comprehensibility of elements in a system model. This entails if the element's naming and decomposition align with its context, if its abstraction level fits the system under review (SUR), and if it is understandable and useful within the context given by the SUR.

The definition of model quality also switches for technical domains and applications [29]. Current tools fail to support practitioners in such sophisticated quality reviews [32]. Therefore, we present an automated quality assessment approach that considers the expert domain knowledge of system engineers. It aims to provide context-, domain- and project-dependent assessments. We identify a lack of research and tool support in performing tailored assessments automatically. We try to close the gap by investigating a data-driven approach and presenting an initial tool prototype that

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MODELS Companion '24, September 22–27, 2024, Linz, Austria

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0622-6/24/09

<https://doi.org/10.1145/3652620.3688339>

combines metrics extracted from model artifact version histories with expert ratings to provide higher-level quality assessments.

Section 2 contains the problem statement, the research questions, and the contribution. Section 3 describes the methodology with our system model context and the metric and quality assessments. We present the results in Section 4 with the quality estimator development pipeline, data processing, training, and estimator evaluation. Section 5 discusses other approaches for model quality management in systems engineering. The conclusion (see Section 6) covers a wrap-up with limitations and future work.

2 Problem Statement

ISO/IEC/IEEE 42010:2022 [20] guidelines and the International Council on Systems Engineering (INCOSE) suggest that system engineers should frequently assess modeling artifact quality [18].

However, the existing guidelines lack pragmatic assessment procedures that are easy to apply [12]. Because of shorter time-to-market and increasing system complexity, frequent model checks often fail due to prohibitive costs.

Manual reviews are time- and cost-intensive and in addition the subjectivity of the topic makes it hard to quantify comprehensibility, readability, or clarity automatically. State-of-the-art quality assessments are rule-based approaches, often referred to as validation suites (e.g., IncQuery Validator [23]). Such suites focus primarily on syntax and completeness of model artifacts. They can ensure, e.g., tracing between requirements and components or naming conventions. Another example is the use of the Object Constraint Language (OCL) [15] to enforce the existence of model elements or types that complement each other [14]. OCL and structured expressions enable the automation of rule checking. To create validation suites, system engineers need to be familiar with the rule specification language and know how to encode model quality requirements in such a formal representation [14].

Another limitation of rule-based approaches is that they cannot adequately capture expert and domain-specific knowledge [16, 29]. Domain-specific languages carry domain knowledge but come with grammatical restrictions. In most cases, they can prevent those errors that rule-based quality assessment approaches are target at. We therefore assert that for a comprehensive analysis of a system model, it is essential to include the expertise of systems engineers familiar with the specific project and its domain. Due to their cost, sophisticated model reviews are only infrequently scheduled before important roll-outs, which is why such assessments do not contribute to the ongoing development of the model artifacts. We explore if we can conduct domain-specific quality assessments through extraction of metrics in model version histories. We want to utilize an automated data-driven approach providing feedback on the comprehensibility of a model considering the results of existing expert assessments on versioned system models.

2.1 Research Questions

We propose a data-driven approach to tackle the aforementioned problem of missing frequent feedback on the artifact quality. To enable an appropriate automated assessment strategy using metric development, we formulate the following research questions:

- RQ1 How can we automatically extract model metrics from system model histories?
- RQ2 How can we collect quality ratings for training an estimator on project and domain-specific quality aspects?
- RQ3 What constitutes an effective training pipeline utilizing model metrics and quality ratings to prototype a quality attribute estimator?

2.2 Contribution

We provide an approach for automated model quality assessments using model metric extraction. We showcase a practical model quality labeling user interface that enables building datasets to train quality estimators. Our analysis of related work, as detailed in Section 5, reveals that the current state-of-the-art does not leverage model version histories for quality attribute estimations.

To address this gap, we work towards a data-driven approach that learns from expert quality ratings of model revisions [5] (focus: model comprehensibility) and the corresponding model metrics. We present work-in-progress results on the applicability of our approach with prototypes of a metric extraction and quality labeling plugin for the Eclipse-based open-source AutoFOCUS3 [9] tool, and of a quality estimator training pipeline. We leverage the version history of assisted driving functions models created during a university lab course conducted in the fortiss Mobility Lab [10] for training the estimator, and for evaluating its accuracy.

3 Approach

We describe here the context of our SUR, the model element types as well as the metrics and quality attributes we use in this study. We also illustrate the implemented quality rating procedure to gather ground truth data for our estimation approach and describe the methodology to derive a working estimator.

3.1 System Model Version History used for Quality Estimator Training and Evaluation

We investigated a model version history with 103 revisions stemming from the development of assisted driving functions on the scale 1:10 fortissimo rover platform [10]. Within a six-month practical course in collaboration with the Technical University of Munich, a team of four students used the AutoFOCUS3 [9] modeling tool to develop a system model that specifies the rover's HW/SW architecture and its behavior. The team developed driver assistance functions (automated lane keeping, and automatic cruise control) and a manual driving function for steering using a remote control, whereby it used a Git repository for revision control of the model. In this study, we focus on the history of the logical architecture [34] since it is the most developed viewpoint in the available model history. It describes the system's architecture and behavior in an implementation-independent way as a hierarchical network of *logical components* whose typed input and output *ports* are connected by directed *channels*. The behavior of atomic leaf components is defined using a *state automaton* or a textual *code specification* [2]. Figure 2 shows a screenshot of a logical component's diagram view.

3.2 Model Metrics

Table 1 depicts the metrics that are extracted for training the quality estimator for each logical component of the investigated models. The element in the focus of the metric extraction or comprehensibility estimation is denoted as the element under review (EUR). The selection of metrics is driven by the metamodel of the AutoFOCUS3 logical architecture that we use in our experiment. We argue that the *Element*, *Port*, *Structure* and *Diagram Metrics* are generally applicable to component-based modeling languages, and the *Code Metrics* are suitable for any textual specification language.

Additionally, we extract descriptive (meta-)information for each element, such as timestamp, project name, Git-commit hash, element name, and identifier.

3.3 Quality Ratings

After reviewing related work [4, 13, 16] and examining quality assessment guidelines from the INCOSE Systems Engineering Handbook [18], we developed five comprehensibility ratings that contribute to assessing the quality of a model. These ratings are designed to evaluate the intuitive ("at first glance") comprehensibility of the EUR, encouraging system engineers to integrate their expert and domain-specific knowledge into the assessment process. The quality ratings range from one to three, where one indicates poor quality, two suggests room for improvement, and three denotes no visible quality degradation. These ratings aim to quantify the comprehensibility in terms of the following dimensions:

- (1) **graphical element representation**: Does the visualization of the EUR represent its impact and role for the SUR?
- (2) **graphical data flow**: Does the graphical representation convey an intuitive understanding of the data flow?
- (3) **element naming**: Do the element names fit in the context of the EUR, and do they convey an intuitive understanding of the element's functionality?
- (4) **interface representation**: Are the port and channel names and the data types comprehensible in the EUR's context?
- (5) **abstraction level**: Is the decomposition of the EUR comprehensible, and are its constituent elements at the expected granularity level?

To conduct the rating, we went through the existing model versions of the rover model and graded each non-atomic logical component according to the five criteria above.

4 Results

Figure 1 illustrates the concept of our process for building an estimator for the comprehensibility of logical components. Using the Eclipse eGit API, our prototype automatically checks out each revision of the model and extracts the 19 metrics listed in Table 1, plus three dimensions to precisely identify model elements by element ID, commit hash, and element name. In addition to the metric extraction, the quality plugin provides a graphical user interface that enables rating logical components with our comprehensibility ratings. Both the metrics and the annotated ratings are stored in a database, from where they are passed to the data processing pipeline. The approach enables the exploration of model quality estimators, splitting the available data into a training and a test set.

We generate linear and non-linear prediction models from the training set and compare them according to established metrics such as mean square error, R^2 score, or accuracy on our test set. Our pipeline proposes the prediction model that performs best with the highest accuracy for the given data. Our code for estimator exploration can later be integrated into the system modeling tool to complete the automatic pipeline. This lowers the hurdle for system engineers who do not want to engage in training prediction models but can benefit from a tailored automated quality feedback facility. For providing feedback on the comprehensibility of the current model version, our concept foresees that the model metrics are computed on the fly and passed to the trained prediction model.

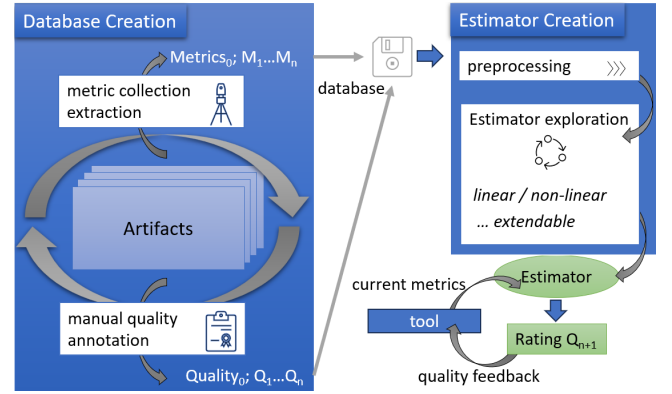


Figure 1: Quality estimator overview with data extraction, creation, and runtime prediction.

4.1 Extracting model metric histories and expert labeling (RQ1, RQ2)

We use the eGit API to check out each model version in the AutoFOCUS3 modeling environment and to extract the metrics listed in Table 1. We store all metrics in an SQLite database. In our experiment, we in total collect 25,342 entries of model elements of our system model versions. The prototype tooling offers an extensible interface for integrating new metrics to be extracted from an existing model version history. When the model is extended, new metrics are computed selectively by checking the already existing commit hashes in the database and performing a new extraction on newly added model versions.

Not all 25,342 model elements are investigated in this study. We focus on the logical components which sum up to 6,412 components over 103 model versions. Our quality ratings incorporate graphical comprehensibility assessments that are based on the graphical representation of the EUR's internal structure (in form of a schematics diagram). By definition of our quality rating process, only non-atomic (non-leaf) elements are rated since the metrics and ratings refer also to the element's internal structure and are therefore not applicable to (atomic) leaf components. Therefore the number of labeled elements reduces to 1,837. The labeling of each element took an approximate time of around one minute to assess the visible UI elements within a logical component which results in an approximated time of 30 hours of labeling. Due to time constraints

Name	Description	T.
#Contained Elements	Number of direct logical component children of the element under review (EUR).	Element M. [11]
#Total Elements	Number of logical components in the EUR's sub-tree including itself.	
#Total Leaf Elements	Number of leaf elements in the EUR's sub-tree.	
Nesting Level	The Hierarchy Level on which the EUR is located.	
#Total Commentable Elements	Number of elements that can have a comment in the EUR's sub-tree.	
#Total Commented Elements	Number of elements that have a comment in the EUR's sub-tree.	Port Metrics
#Ports	Number of input and output ports of the EUR.	
#Total Ports	Number of ports in the EUR's sub-tree.	
#Total Input Ports	Number of input ports in the EUR's sub-tree.	
#Total Output	Number of Output ports in the EUR's sub-tree.	Struct. M.
#Channels	Number of channels in the EUR.	
Clustering Coefficient	Graph-theoretic measure depicting how the EUR's direct children cluster together [17].	
Density	Ratio of Connections in the EUR and the maximum number of Connections [31].	Diag.
Surface Coverage	Percentage of covered area in a EUR's diagram view (within its min. bounding box) [26].	
Deviation of Channel lengths	Standard deviation of channel lengths in the EUR diagram [31].	Code M. [38]
#Total Code Lines	Number of code lines in the EUR's sub-tree.	
#Total Code Comments	Number of code comments in the EUR's sub-tree.	
#Total Code Constants	Number of code constants in the EUR's sub-tree.	
Total Code Cyclomatic Complexity	Cyclomatic complexity of code in the EUR's sub-tree.	

Table 1: Model element metrics for component-based model languages and text-based specification languages.

the labeling process was done by a single researcher for the pipeline initialization.

For the labeling process, we apply the model quality plugin integrated into AutoFOCUS3 in the frame of this work that enables to rate our logical architecture elements (see Figure 2). To enable rating historic model versions, the plugin checks out a specific commit of the model version history into the tool's workspace. The control on the right-hand side of the plugin offers a UI to set quality labels for the model element that is currently selected in the model's diagram editor. The star widgets represent the current rating of the visualized element describing the comprehensiveness of its internal structure, visualization, element decomposition, names, channels and ports. Therefore the possible rating values are one, two and three.

4.2 Pre-Processing and Metric Data Analysis

To understand the data and prepare it for estimator training, the SQLite database is read into a Jupyter notebook. The analysis revealed that the mean number of annotated non-leaf model elements per logical architecture is 17.8 per version. Adding leaf elements looking at the total number of logical components per version, we reach around 89.

Because of element characteristics, some elements do not support the extraction of all metrics. We can extract 11 metrics to 100%. For code specification metrics, we achieve a completion of 62.25%. They only apply to EURs that have code-based leaves in their sub-tree. The metric describing the number of total commented elements in a sub-tree is extracted in 45.67% of the annotated elements. We engage in a correlation analysis to gather more insights into possible

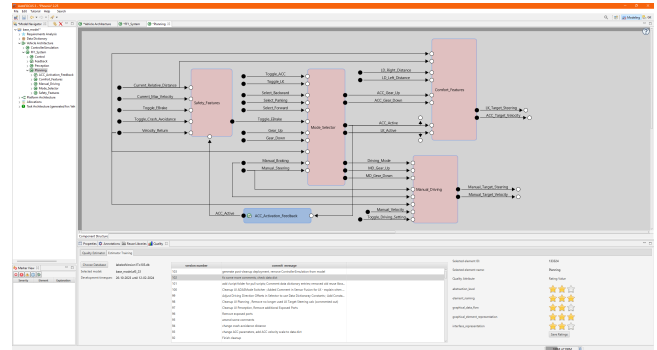


Figure 2: AutoFOCUS3 model quality plugin for comprehensibility rating (bottom right) and Git-revision selection (bottom middle). Top: component diagram editor of EUR.

relations between our metrics and quality ratings. We exclude non-numeric and constant dimensions and drop rows with missing values. We further remove outliers with a 0.15 quantile and 0.85 quantile, apply min-max scaling, and calculate correlation matrices with Pearson [33], Spearman [37] and Kendall [21] correlation coefficients.

The correlation analysis shows that none of our extracted metrics highly correlates with one of our annotated comprehensibility ratings.

4.3 Estimator Training (RQ3)

The best linear regressor predicts the quality of element naming within the EUR with four metrics: #Contained Elements, #Total

Code Comments, Deviation of Channel Lengths, and Nesting Level with a mean squared error of 0.30 and a R-Square value of 0.32. This indicates an insufficient linear relationship between metrics and labels also for multi-parameter regression approaches.

We use a random-forests as a non-linear approach. We explore suitable subsets of metrics by comparing the accuracy of possible combinations as we did for the linear models. The accuracy is measured after training on a test data set, which uses 20% of the total 1837 entries. The incompleteness of features can lead to a loss of training data. To reduce the data loss for incompletely extracted metrics, we remove the group of code-based metrics and the #Total Commented Elements metric. We adapt the data processing by removing the normalization as it is not relevant for random forest creation.

With a maximum number of five metrics per estimator and fourteen metrics in the exploration, we check therefore

$$\binom{14}{5} + \binom{14}{4} + \binom{14}{3} + \binom{14}{2} + \binom{14}{1} = 3472$$

combinations. For each of the five quality ratings, we output the best random-forest estimator in Table 2.

Predictors	Rating	Acc.
Nesting Level, #Channels, Surface Coverage, Deviation of Channel Lengths	Abstraction Level	0.94
Nesting Level, #Ports, #Total Ports, Density, Surface Coverage	Element Naming	0.92
Nesting Level, #Contained Elements, #Total Elements, Deviation of Channel Lengths	Graphical Data Flow	0.94
Clustering Coefficient, #Channels, #Total Elements, Deviation of Channel Lengths	Graphical Element Representation	0.95
Nesting Level, #Total Ports, #Total Leaf Elements, Surface Coverage	Interface Representation	0.94

Table 2: Best accuracy ("Acc.") random forest estimators for the ratings with at most five features.

The feature importance analysis shows that graphical metrics such as the Surface Coverage and Deviation of Channel Lengths have the highest importance value. The Surface Coverage ranges from 0.3 for the abstraction level estimation to 0.6 for the interface representations. The Deviation of Channel Lengths shows the highest feature importance for the graphical data flow rating and the graphical element representation rating at around 0.45.

To extend the exploration of estimation models, we started investigating support vector machines (SVM), but could not identify a better performance within our first experiments. The best-performing SVM resulted in a 0.79 accuracy value for the estimation of the graphical element representation rating.

5 Related Work

To elaborate on the targeted research gap, we review related work that aims at improving model quality.

Giraldo *et al.* [13] collected quality model definitions in the MBSE context. Examples include quality models in model-driven web development [6], the SEQUAL framework [22], the 6C framework [29], UML guidelines as specified in ISO 9126 and newer versions (see [1]) or model size metrics (MoSMa) [24].

Basciani *et al.* investigated quality models and their application on model artifacts in MBSE. The quality assessment strategies cover relationships between different artifacts and transformation-based approaches. Examples of quality attributes used within the models are coverage or transformation information loss. The attribute values are calculated via pre-defined formulas with a combination of the extracted metrics [4].

From an industry perspective, the IncQuery Group provides a validator for Enterprise Architect [19] that provides a more sophisticated rule-based approach. It offers rule sets that are compliant with existing standards such as ISO 26262 or Automotive Spice and that can be further customized. It also extends rule sets with an additional quality score that improves visualization for practitioners to detect quality degradation more easily [23].

All of the previously mentioned work engages in quality assessment but does not include any analysis of the model history.

Research that targets model quality assessments on system model artifacts often focuses on taxonomy or terminology definitions [13, 29]. The more practical papers cover rule or metric-based assessments, which in contrast to our work cannot capture project- or domain specific quality properties.

Popoola describes the evolution of model smells in model artifact histories. His work provides insights into when and how model smells are introduced and what might be the decision-making behind their introduction [35]. With our work, we contribute to connecting the analysis of model artifact version histories (by means of model metrics) with review-based model quality assessments.

6 Conclusion and Future Work

This paper presents a concept for quality attribute estimation based on system model artifact metric histories. We introduce a metric extraction and comprehensibility labeling pipeline with estimator exploration and training to initiate a project and domain-specific automated quality rating.

Future work will extend metric extraction and quality ratings to capture model elements with higher details. We will investigate quality ratings beyond comprehensibility covering potential project and domain-specific assessments to evaluate our approach using industrial systems engineering models. To reduce subjectivity we will conduct more thorough labeling by different researchers in future work and evaluate the approach in comparison to rule-based approaches.

To investigate scalability, we will first increase the degree of automation for training and initializing the model quality estimator and try to achieve a low entry hurdle for practitioners.

Acknowledgments

This work was in parts funded by the German Federal Ministry of Education and Research (BMBF) in the KOMET project (grant no. 01IS24012B) and the Federal Ministry for Economic Affairs and Climate Action (BMWK) in the TASTE project (grant no. 16THB0009C).

References

- [1] ISO/IEC JTC 1/SC 7. 2024. Innovation management – Innovation management system – Guidance. <https://www.iso.org/standard/78175.html>
- [2] Vincent Aravantinos, Sebastian Voss, Sabine Teufl, Florian Hölzl, and Bernhard Schätz. 2015. AutoFOCUS 3: Tooling Concepts for Seamless, Model-based Development of Embedded Systems. In *Proc. 8th Int. Workshop Model-based Architecting of Cyber-Physical and Embedded Systems (ACES-MB)*. 19–26.
- [3] Paris Avgeriou, Philippe Kruchten, Ipek Ozkaya, and Carolyn Seaman. 2016. Managing Technical Debt in Software Engineering. *Dagstuhl Reports* 6, 4 (2016), 110–138. <https://doi.org/10.4230/DagRep.6.4.110>
- [4] Francesco Basciani, Juri Di Rocco, Davide Di Ruscio, Ludovico Iovino, and Alfonso Pierantonio. 2019. A tool-supported approach for assessing the quality of modeling artifacts. *Journal of Computer Languages* 51 (2019), 173–192. <https://doi.org/10.1016/j.col.2019.02.003>
- [5] Konstantin Rupert Blaschke. 2024. Automated Model Quality Estimation and Change Impact Analysis on Model Histories. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (Lisbon, Portugal) (ICSE-Companion '24)*. Association for Computing Machinery, New York, NY, USA, 153–155. <https://doi.org/10.1145/3639478.3639809>
- [6] F.J. Domínguez-Mayo, M.J. Escalona, M. Mejias, M. Ross, and G. Staples. 2012. Quality evaluation for Model-Driven Web Engineering methodologies. *Information and Software Technology* 54, 11 (2012), 1265–1282. <https://doi.org/10.1016/j.infsof.2012.06.007>
- [7] Marcela G. dos Santos, Bianca M. Napoleão, Fabio Petrillo, Darine Ameyed, and Fehmi Jaafar. 2020. A Preliminary Systematic Mapping on Software Engineering for Robotic Systems: A Software Quality Perspective. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops (IC-SEW'20)*. Association for Computing Machinery, New York, NY, USA, 647–654. <https://doi.org/10.1145/3387940.3392197>
- [8] John Fitzgerald, Carl Gamble, Peter Gorm Larsen, Kenneth Pierce, and Jim Woodcock. 2015. Cyber-Physical Systems Design: Formal Foundations, Methods and Integrated Tool Chains. In *2015 IEEE/ACM 3rd FME Workshop on Formal Methods in Software Engineering*. IEEE, Florence, Italy, 40–46. <https://doi.org/10.1109/FormalISE.2015.14>
- [9] fortiss. 2024. AutoFOCUS3. <https://af3.fortiss.org/>. Accessed: 2024-07-15.
- [10] fortiss. 2024. Mobility Lab. <https://www.fortiss.org/en/research/fortiss-labs/detail/mobility-lab/>. Accessed: 2024-07-15.
- [11] Marcela Genero, Mario G. Piattini, and Coral Calero. 2005. A Survey of Metrics for UML Class Diagrams. *J. Object Technol.* 4 (2005), 59–92.
- [12] Fáber D. Giraldo, Ángela J. Chicaiza, Sergio España, and Óscar Pastor. 2021. Empirical validation of a quality framework for evaluating modelling languages in MDE environments. *Software Quality Journal* 29, 2 (2021), 275 – 307. <https://doi.org/10.1007/s11219-021-09554-1>
- [13] Fáber D. Giraldo, Sergio España, and Oscar Pastor. 2014. Analysing the concept of quality in model-driven engineering literature: A systematic review. In *2014 IEEE Eighth International Conference on Research Challenges in Information Science (RCIS)*. IEEE, Marrakech, Morocco. <https://doi.org/10.1109/RCIS.2014.6861030>
- [14] Carlos A. González and Jordi Cabot. 2014. Formal verification of static software models in MDE: A systematic review. *Information and Software Technology* 56, 8 (2014), 821–838. <https://doi.org/10.1016/j.infsof.2014.03.003>
- [15] Object Management Group. 2014. Object Constraint Language Specification Version 2.4. <https://www.omg.org/spec/OC/L/2.4/About-OC.L>.
- [16] Myron Hecht and Jaron Chen. 2021. Verification and Validation of SysML Models. *INCOSE International Symposium* 31, 1 (2021), 599–613. <https://doi.org/10.1002/j.2334-5837.2021.00857.x>
- [17] Paul W. Holland and Samuel Leinhardt. 1971. Transitivity in Structural Models of Small Groups. *Comparative Group Studies* 2, 2 (1971), 107–124. <https://doi.org/10.1177/104649647100200201>
- [18] INCOSE. 2015. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities* (4th ed.). Wiley-Blackwell, San Diego, CA, USA.
- [19] IncQuery Group. 2024. IncQuery Validator for Enterprise Architect. <https://incquery.io/validator-for-enterprise-architect>. Accessed: 2024-07-15.
- [20] ISO/IEC/IEEE. 2022. Systems and Software Engineering – Architecture Description. Standard No. ISO/IEC/IEEE 42010:2022. <https://www.iso.org/standard/74393.html>
- [21] M. G. Kendall. 1938. A New Measure of Rank Correlation. *Biometrika* 30, 1/2 (1938), 81–93. <http://www.jstor.org/stable/2332226>
- [22] John Krogstie. 2012. *Quality of Models*. Springer London, London, 205–247. https://doi.org/10.1007/978-1-4471-2936-3_4
- [23] Géza Kulcsár, Johannes Kristan, and Sven Erik Jeroschewski. 2022. Correctness, Completeness, Consistency: Challenges of Migrating Arrowhead System-of-Systems Models. In *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*. IEEE Press, Budapest, Hungary, 1–6. <https://doi.org/10.1109/NOMS4207.2022.9789830>
- [24] Christian F. J. Lange. 2007. Model Size Matters. In *Models in Software Engineering*, Thomas Kühne (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 211–216.
- [25] Yu Liu, Dong Wang, Jinhua Mi, and He Li (Eds.). 2023. *Advances in Reliability and Maintainability Methods and Engineering Applications* (1 ed.). Springer Cham, Cham, Switzerland. XXXVII, 622 pages. <https://doi.org/10.1007/978-3-031-28859-3>
- [26] Zipeng Liu, Takayuki Itoh, Jessica Q. Dawson, and Tamara Munzner. 2020. The Sprawlter Graph Readability Metric: Combining Sprawl and Area-Aware Clutter. *IEEE Transactions on Visualization and Computer Graphics* 26, 6 (June 2020), 2180–2191. <https://doi.org/10.1109/TVCG.2020.2970523>
- [27] Törngren M. and Sellgren U. 2018. Complexity Challenges in Development of Cyber-Physical Systems. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 10760 LNCS (2018), 478 – 503. https://doi.org/10.1007/978-3-319-95246-8_27
- [28] Mohamed M.A., Challenger M., and Kardas G. 2020. Applications of model-driven engineering in cyber-physical systems: A systematic mapping study. *Journal of Computer Languages* 59 (2020). <https://doi.org/10.1016/j.col.2020.100972>
- [29] Parastoo Mohagheghi, Vegard Dehlen, and Tor Neple. 2009. Definitions and approaches to model quality in model-based software development – A review of literature. *Information and Software Technology* 51, 12 (2009), 1646–1669. <https://doi.org/10.1016/j.infsof.2009.04.004> Quality of UML Models.
- [30] Mustafa Abshir Mohamed, Geylani Kardas, and Moharram Challenger. 2021. Model-Driven Engineering Tools and Languages for Cyber-Physical Systems: A Systematic Literature Review. *IEEE Access* 9 (2021), 48605–48630. <https://doi.org/10.1109/ACCESS.2021.3068358>
- [31] Gavin J. Mooney, Helen C. Purchase, Michael Wybrow, and Stephen G. Kobourov. 2024. The Multi-Dimensional Landscape of Graph Drawing Metrics. In *2024 IEEE 17th Pacific Visualization Conference (PacificVis)*. 122–131. <https://doi.org/10.1109/PacificVis60374.2024.00022>
- [32] Mert Ozkaya. 2019. Are the UML modelling tools powerful enough for practitioners? A literature review. *IET Software* 13, 5 (2019), 338–354. <https://doi.org/10.1049/iet-sen.2018.5409>
- [33] Karl Pearson. 1896. VII. Mathematical contributions to the theory of evolution.—III. Regression, heredity, and panmixia. *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character* 187 (1896), 253–318. <https://doi.org/10.1098/rsta.1896.0007>
- [34] Klaus Pohl, Harald Hönninger, Reinhold Achatz, and Manfred Broy. 2012. *Model-Based Engineering of Embedded Systems* (1 ed.). Springer Berlin, Heidelberg, Berlin, Germany. XII, 304 pages. <https://doi.org/10.1007/978-3-642-34614-9> Published: 09 November 2012 (Hardcover), 08 November 2012 (eBook), 14 December 2014 (Softcover).
- [35] Saheed Popoola. 2021. *Change Analysis Across Version Histories of Systems Models*. Ph.D. Dissertation. The University of Alabama. <http://ir.ua.edu/handle/123456789/8198>
- [36] D.K.K. Shyamal, P.P.G.D. Asanka, and D. Wickramaarachchi. 2023. A Comprehensive Approach to Evaluating Software Code Quality Through a Flexible Quality Model. In *2023 International Research Conference on Smart Computing and Systems Engineering (SCSE)*, Vol. 6. IEEE, Kelaniya, Sri Lanka, 1–8. <https://doi.org/10.1109/SCSE59836.2023.10215004>
- [37] C. Spearman. 1904. The Proof and Measurement of Association between Two Things. *The American Journal of Psychology* 15, 1 (1904), 72–101. <http://www.jstor.org/stable/1412159>
- [38] Martin Vogel, Peter Knapik, Moritz Cohrs, Bernd Szyperrek, Winfried Pueschel, Haiko Etzel, Daniel Fiebig, Andreas Rausch, and Marco Kuhrmann. 2021. Metrics in automotive software development: A systematic literature review. *Journal of Software: Evolution and Process* 33, 2 (2021), e2296. <https://doi.org/10.1002/smr.2296>
- [39] Danny Weyns, Tomas Bures, Radu Calinescu, Barnaby Craggs, John Fitzgerald, David Garlan, Bashar Nuseibeh, Liliana Pasquale, Awais Rashid, Ivan Ruchkin, and Bradley Schmerl. 2021. Six Software Engineering Principles for Smarter Cyber-Physical Systems. In *2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*. IEEE, DC, USA, 198–203. <https://doi.org/10.1109/ACSOS-C52956.2021.00058>
- [40] Xi Zheng and Christine Julien. 2015. Verification and Validation in Cyber Physical Systems: Research Challenges and a Way Forward. In *2015 IEEE/ACM 1st International Workshop on Software Engineering for Smart Cyber-Physical Systems*. IEEE, Florence, Italy, 15–18. <https://doi.org/10.1109/SESCPS.2015.11>

Received 15 July 2024; accepted 7 August 2024